



Sequential Recommendation: From Markov Transitions to Generative Retrieval

MINDS Lab.

**Undergraduate Research Intern
HoonUi Lee**

2026-08-05

Contents

- ❖ Sequential Recommendation
- ❖ Research Before SASRec
- ❖ Self-Attention-based Sequential Recommendation
- ❖ Recommender Systems with Generative Retrieval

❖ KDD 논문들을 살펴보니 Sequential Recommendation 관련 연구가 많았음

Title
FuXi-Linear: Unleashing the Power of Linear Attention in Long-term Time-aware Sequential Recommendation
Align-for-Fusion: Harmonizing Triple Preferences via Dual-oriented Diffusion for Cross-domain Sequential Recommendation
Sharpness-aware Model Merging with salience recovery for LLM-based Cross-Domain Sequential Recommendation
Mitigating Popularity Bias Amplification in Scaling Transformer-based Sequential Recommenders
MergeRec: Model Merging for Data-Isolated Cross-Domain Sequential Recommendation
LISRec: Modeling User Preferences with Learned Item Shortcuts for Sequential Recommendation
From Agnostic to Specific: Latent Preference Diffusion for Multi-Behavior Sequential Recommendation
Align-for-Fusion: Harmonizing Triple Preferences via Dual-oriented Diffusion for Cross-domain Sequential Recommendation

- ❑ 개별 논문을 읽기 전, 공통 배경을 이해할 필요가 있음
- ❑ 이번 발표에서는 다음 두 질문을 다룰 예정
 1. 사용자의 행동 순서를 어떻게 모델링해 왔는가?
 2. 최근에는 왜 아이템 ID를 생성하는 방식까지 등장했는가?

Sequential Recommendation

- ❑ What Does a Recommender System Predict?
- ❑ Why Sequential Recommendation?
- ❑ Problem Definition

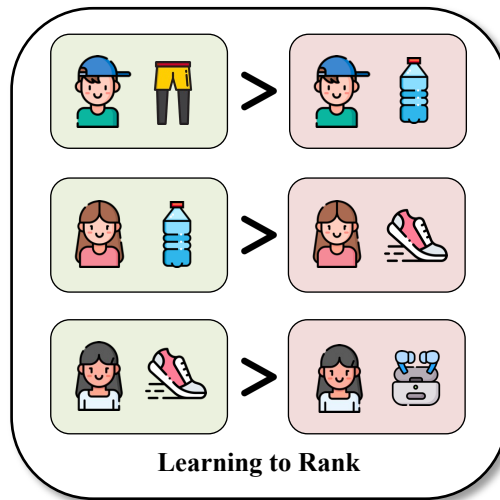
What Does a Recommender System Predict?

Sequential Recommendation

- ❖ Use observed interactions to predict preferences for unseen items or rank candidate items

	?	5	4	4
	4	?	?	5
	1	3	?	3
	5	5	2	?

Matrix Completion

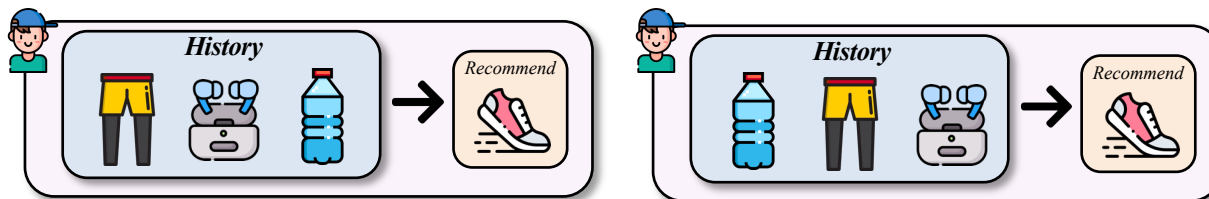


Why Sequential Recommendation?

Sequential Recommendation

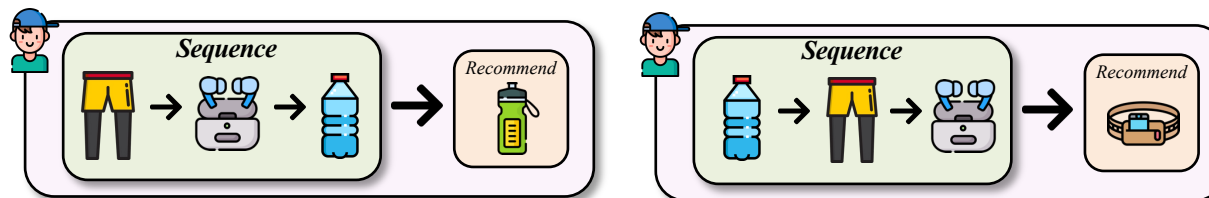
- ❖ Predict the next item using both long-term preferences and the order of recent interactions

Traditional Recommendation



*Interaction history as an **unordered set***

Sequential Recommendation



*Interaction history as an **ordered sequence***

Why Sequential Recommendation?

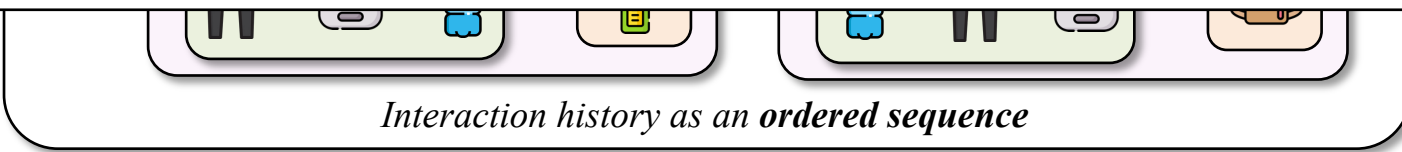
Sequential Recommendation



- ❖ Predict the next item using both long-term preferences and the order of recent interactions

Traditional Recommendation

Sequential recommendation captures user preferences that change over time and with recent behavior



The diagram illustrates the concept of interaction history as an ordered sequence. It shows a horizontal sequence of colored boxes (green, purple, orange) containing various icons (e.g., a person, a book, a shopping cart). These boxes are arranged in a row, with the most recent interaction on the right. The entire sequence is enclosed in a rounded rectangular frame.

Interaction history as an ordered sequence

Problem Definition

Sequential Recommendation

❖ Predict the next item from an ordered interaction history

□ $S_u = (i_1, i_2, \dots, i_t), p(i_{t+1} | i_1, i_2, \dots, i_t)$

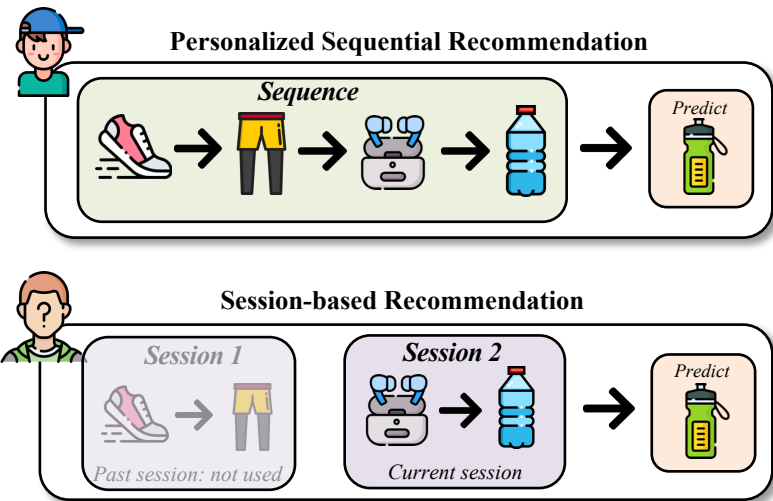
❖ Two common settings

□ Personalized sequential recommendation

- ▶ User-specific long-term history across sessions
- ▶ Long-term preference + current intent

□ Session-based Recommendation

- ▶ Persistent user history may be unavailable
- ▶ Current session only



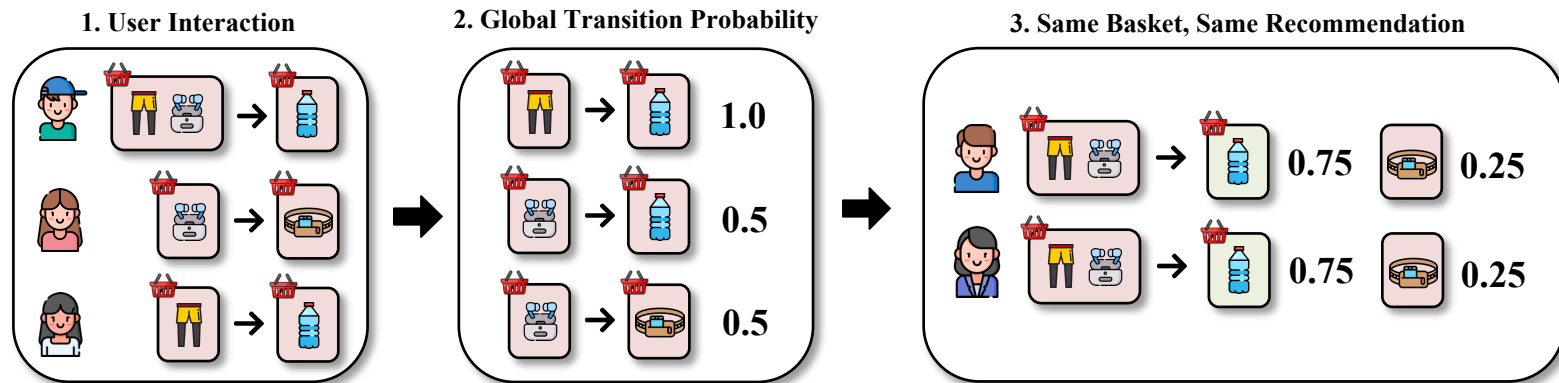
Research Before SASRec

- ❑ Markov Chain-based approaches for next-item prediction
 - Factorizing Personalized Markov Chains for Next-Basket Recommendation (WWW 10')
- ❑ Modeling the entire interaction sequence within a session
 - Session-based Recommendations with Recurrent Neural Networks (ICLR 16')
- ❑ Capturing Diverse Sequential Patterns with CNNs
 - Personalized Top-N Sequential Recommendation via Convolutional Sequence Embedding (WSDM 18')

Markov Chain-based approaches for next-item prediction

Research Before SASRec

❖ A Markov Chain learned from global transition patterns

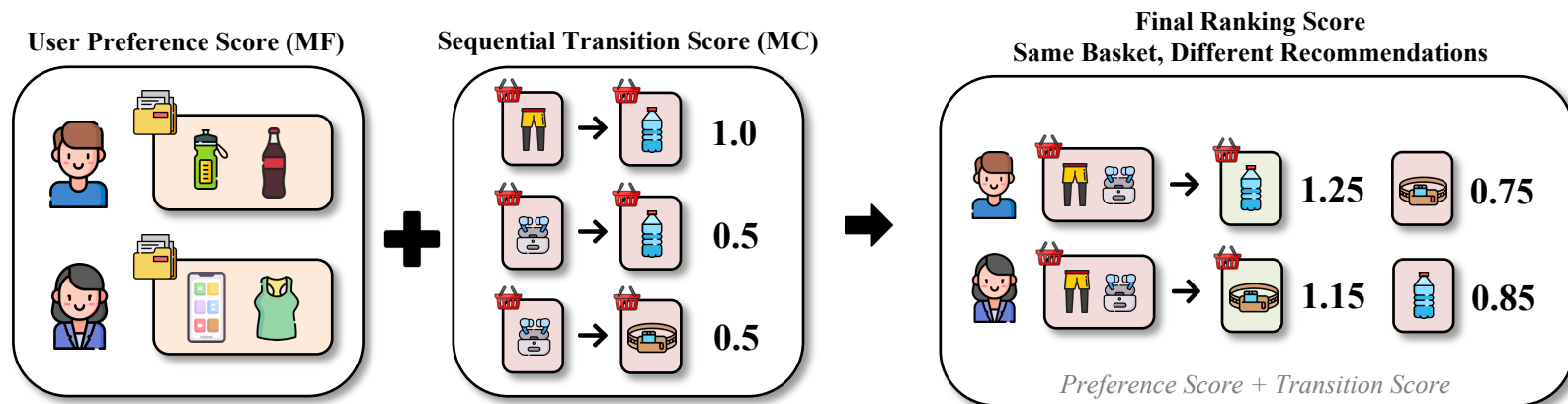


- ❑ Count transitions from items in one basket to items in the next basket for each user
- ❑ Aggregate all users' histories to estimate a shared transition probability $P(i_{\text{next}} | i_{\text{previous}})$
- ❑ Ignore user-specific preferences
 - Users with the same recent basket receive the same recommendations

Factorizing Personalized Markov Chains

Research Before SASRec

❖ Factorizing Personalized Markov Chains for Next-Basket Recommendation (WWW 10')

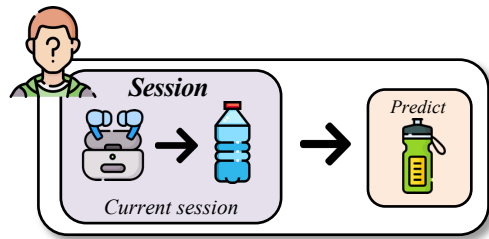


- ❑ Learn long-term user preferences with Matrix Factorization
- ❑ Model sequential transitions from the most recent basket
- ❑ Personalize recommendations even for the same recent basket

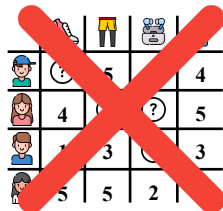
Modeling the entire interaction sequence within a session

Research Before SASRec

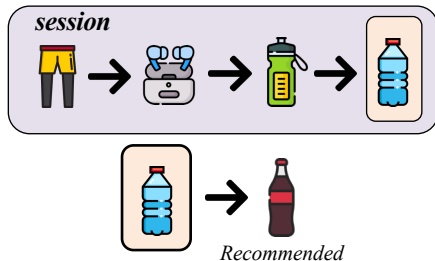
❖ Session-based Recommendations with Recurrent Neural Networks (ICLR 16')



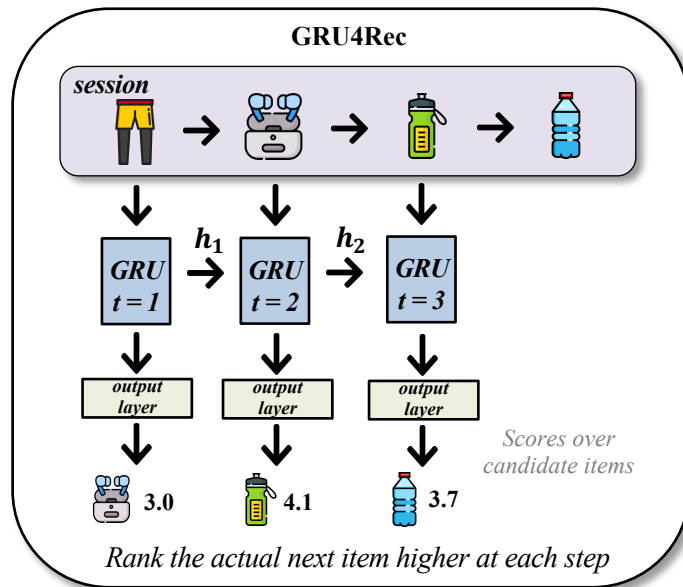
User IDs and long-term interaction histories are often unavailable



Matrix Factorization is difficult to apply



Previous methods mainly rely on the most recent interaction



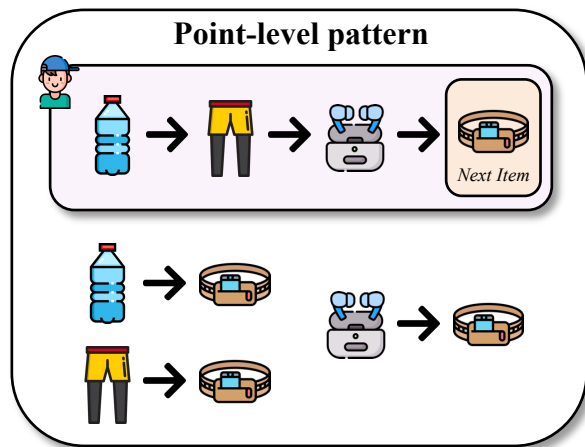
- Accumulate past interactions in the hidden state and rank the next item at each time step

Capturing Diverse Sequential Patterns with CNNs

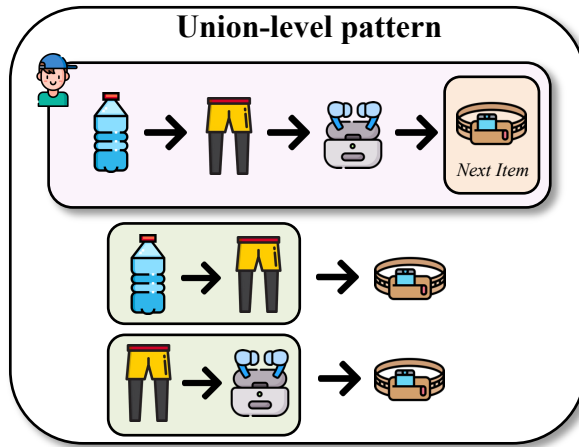
Research Before SASRec

❖ Personalized Top-N Sequential Recommendation via Convolutional Sequence Embedding (WSDM 18')

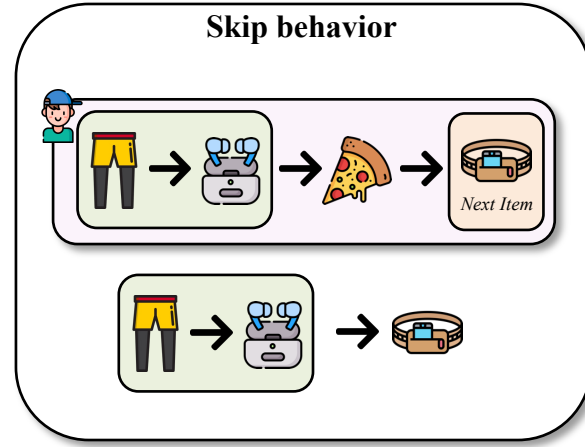
- ❑ Model the effects of past interactions in **three different ways**



- ❑ Individual effects of past items



- ❑ Joint effects of multiple past items

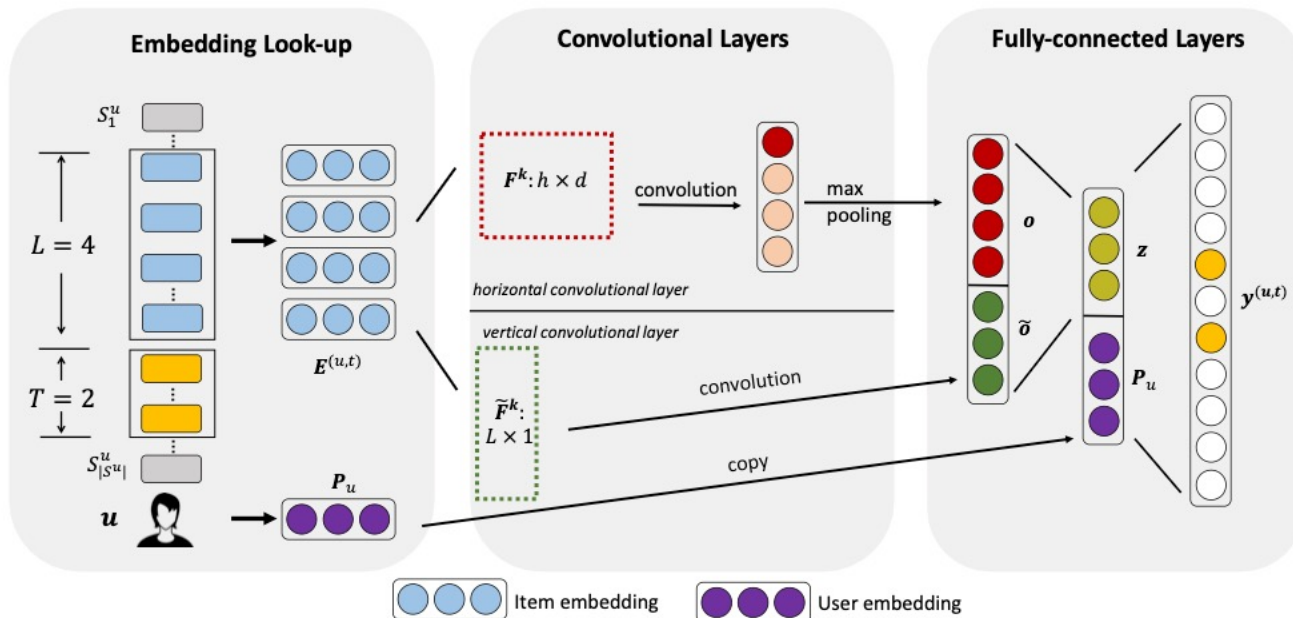


- ❑ Effects across non-adjacent interactions

Capturing Diverse Sequential Patterns with CNNs

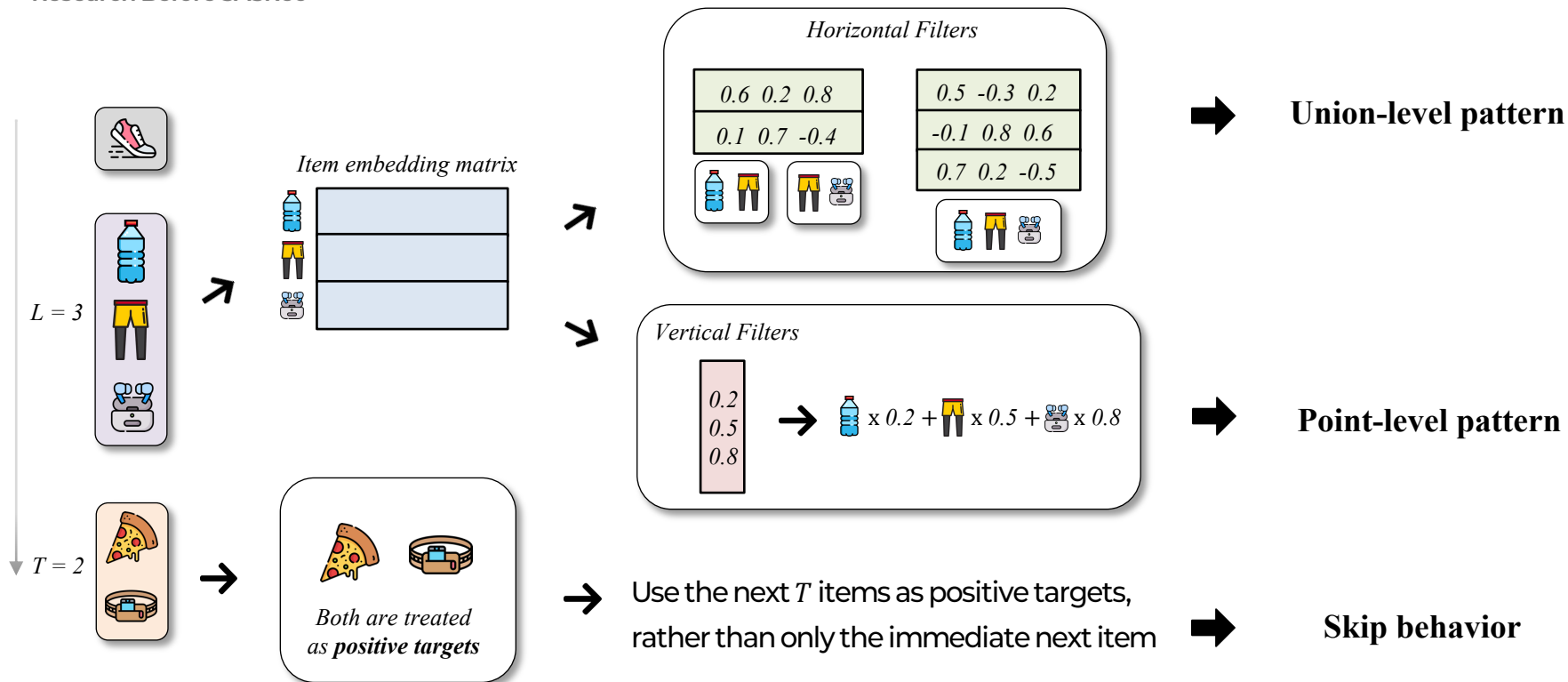
Research Before SASRec

❖ Caser (Convolutional Sequence Embedding Recommendation)



Capturing Diverse Sequential Patterns with CNNs

Research Before SASRec



Research Before SASRec

Model	How Past Interactions are Modeled	Strengths	Key Limitations
FPMC	Recent item transitions	Simple and efficient	Limited long-range dependency modeling
GRU4Rec	Sequential hidden states	Captures the entire session	Sequential computation; early information may fade
Caser	CNN patterns within a fixed window	Captures local and union-level patterns	Limited receptive field

Self-Attention-based Sequential Recommendation (18-ICDM)

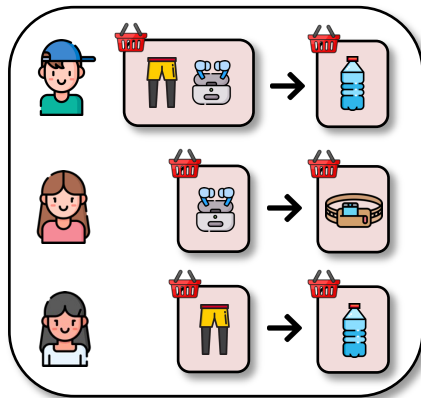
- ❑ Can We Combine the Strengths of Markov Chains and RNNs?
- ❑ Methodology
- ❑ Strengthness & Limitations

Can We Combine the Strengths of Markov Chains and RNNs?

Self-Attention-based Sequential Recommendation

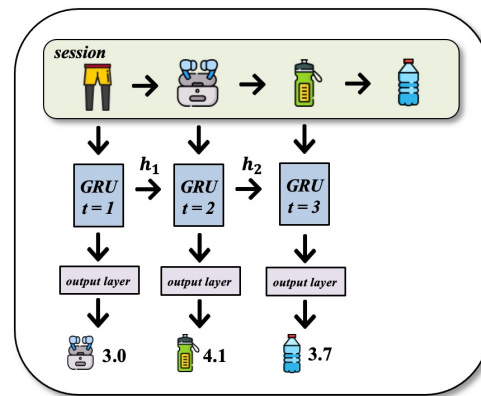
- ❖ Markov Chains and RNNs capture different but complementary sequential signals

Markov Chain



- ❑ Selective Focus on Recent Actions
 - Focuses on one or a few recent actions
 - Simple and effective on sparse data
 - Limited ability to capture long-range patterns

RNN



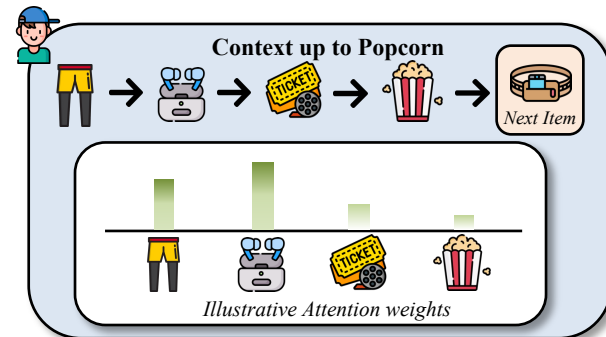
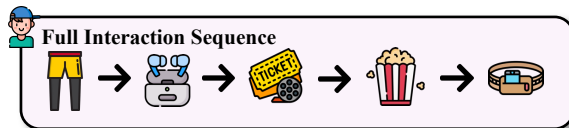
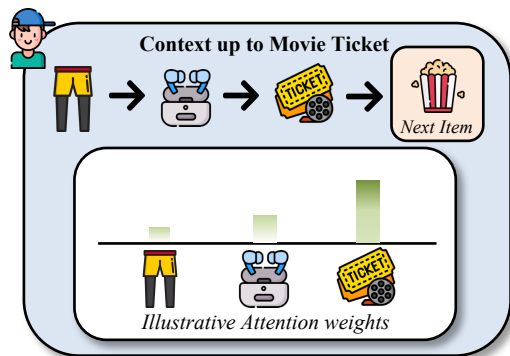
- ❑ Expressive Long-Term Modeling
 - Compresses the entire history into a hidden state
 - Can capture long-term sequential patterns
 - Older information may fade during sequential processing

SASRec: The Best of Both Worlds

Self-Attention-based Sequential Recommendation

❖ SASRec selects different past actions for each prediction step

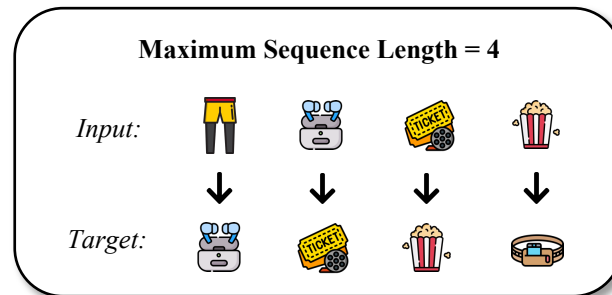
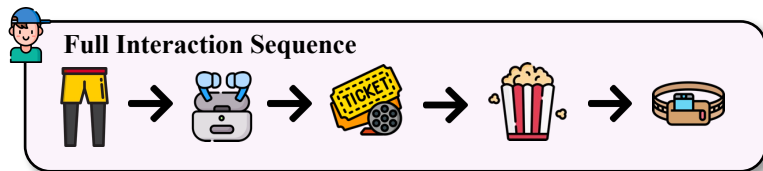
- ❑ Assigns context-dependent attention weights to past actions
- ❑ Accesses the full history without compressing it into a single recurrent state
- ❑ Focuses on recent or distant actions depending on their relevance



Training Data Construction

Self-Attention-based Sequential Recommendation

❖ Train next-item predictions at all sequence positions



- ❑ The input and target sequences are shifted by one position

❖ Embedding Layer

- ❑ Convert each item ID into a d -dimensional vector and add a positional embedding
 - Item Embedding + Position Embedding

$$\hat{\mathbf{E}} = \begin{bmatrix} \mathbf{M}_{s_1} + \mathbf{P}_1 \\ \mathbf{M}_{s_2} + \mathbf{P}_2 \\ \vdots \\ \mathbf{M}_{s_n} + \mathbf{P}_n \end{bmatrix}$$

Self-Attention Block

Self-Attention-based Sequential Recommendation

❖ Self-Attention Layer

- ❑ $\mathbf{S} = \text{SA}(\hat{\mathbf{E}}) = \text{Attention}(\hat{\mathbf{E}}\mathbf{W}^Q, \hat{\mathbf{E}}\mathbf{W}^K, \hat{\mathbf{E}}\mathbf{W}^V)$
 - Captures context-dependent and asymmetric item-to-item relations

❖ Causal Mask to Prevent Future Information Leakage

- ❑ Parallel next-item prediction without future leakage
- ❑ Each position attends only to positions at or before itself

Causal Mask

Q \ K				
	✓	MASK	MASK	MASK
	✓	✓	MASK	MASK
	✓	✓	✓	MASK
	✓	✓	✓	✓

❖ Point-Wise FFN + Block Stacking

- ❑ $\mathbf{F}_i = \text{FFN}(\mathbf{S}_i) = \text{ReLU}(\mathbf{S}_i\mathbf{W}^{(1)} + \mathbf{b}^{(1)})\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$

$$\mathbf{S}^{(b)} = \text{SA}(\mathbf{F}^{(b-1)}),$$

$$\mathbf{F}_i^{(b)} = \text{FFN}(\mathbf{S}_i^{(b)}), \quad \forall i \in \{1, 2, \dots, n\}$$

Prediction Layer

Self-Attention-based Sequential Recommendation

- ❖ Score each candidate item using the final sequence representation



Score Calculation

$$r_{i,t} = \mathbf{F}_t^{(b)} \mathbf{M}_i^T$$



- ❑ Compute the dot-product score between the final representation $\mathbf{F}_t^{(b)}$ at each position and each candidate item embedding
 - $\mathbf{M}_i$: Embedding of candidate item i
 - $r_{i,t}$: Score of item i as the next item at position t
- ❑ Rank candidate items by their scores to generate the next-item prediction or Top- K recommendations

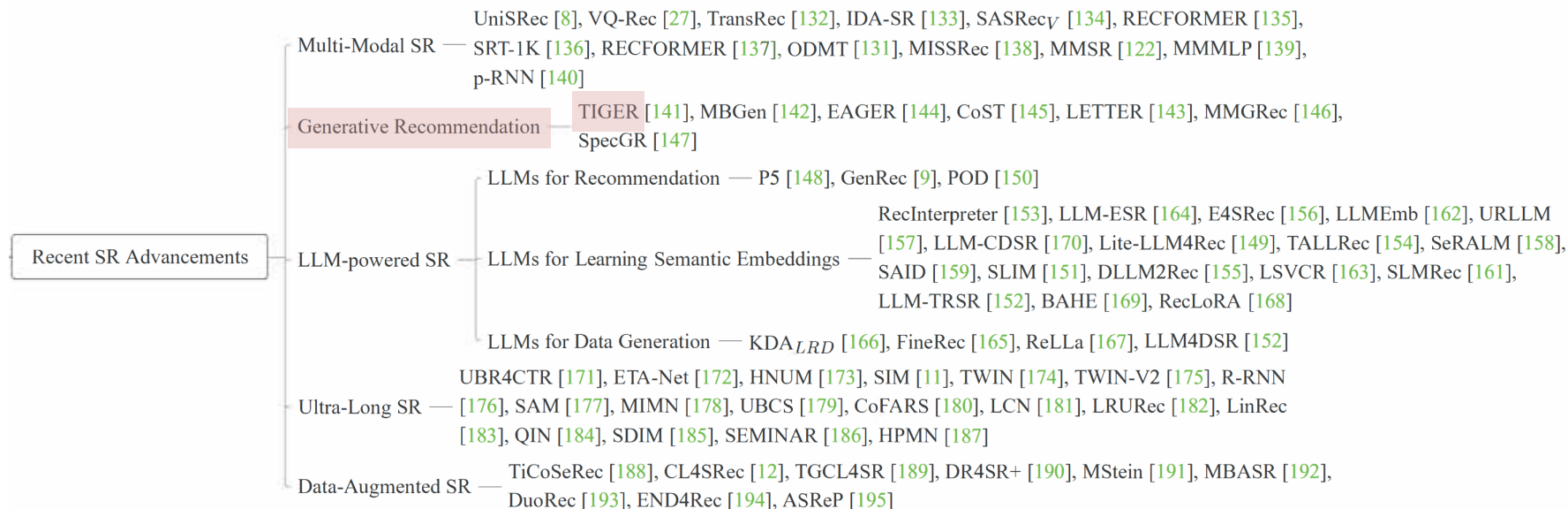
❖ Strengthness

- ❑ Adaptively balances recent context and long-term sequential patterns
 - Combines the local focus of Markov Chains with the long-range modeling of RNNs
- ❑ More parallelizable than RNNs
 - RNNs process positions sequentially
 - Causally masked self-attention processes all sequence positions in parallel during training
- ❑ Personalization without an explicit user embedding
 - Builds a dynamic user representation from the interaction sequence
 - The representation changes as the user's recent behavior changes

❖ Limitations

- ❑ Quadratic attention cost with sequence length
 - Standard self-attention requires $O(n^2)$ computation per layer
- ❑ Models order, but not actual time
 - Does not explicitly use timestamps or time gaps between interactions
 - Requires time-aware extensions
- ❑ ID-based item representations are weak for cold-start items
 - New items have no learned embeddings
 - Rarely interacted items may have poorly learned embeddings
 - Vanilla SASRec does not use item content by default

Recent Sequential Recommendation Research

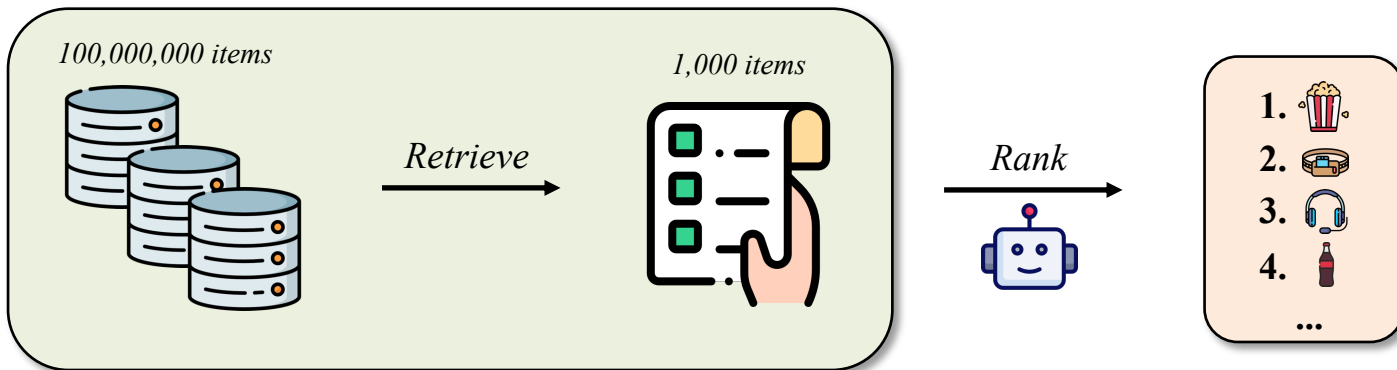


Recommender Systems with Generative Retrieval (NeurIPS 23')

- ❑ Retrieve-and-Rank Strategy in RS
 - From Embedding Retrieval to Identifier Generation
- ❑ How TIGER Implements Generative Retrieval
- ❑ Experiments

Retrieve-and-Rank Strategy in RS

Transformer Index for Generative Recommenders



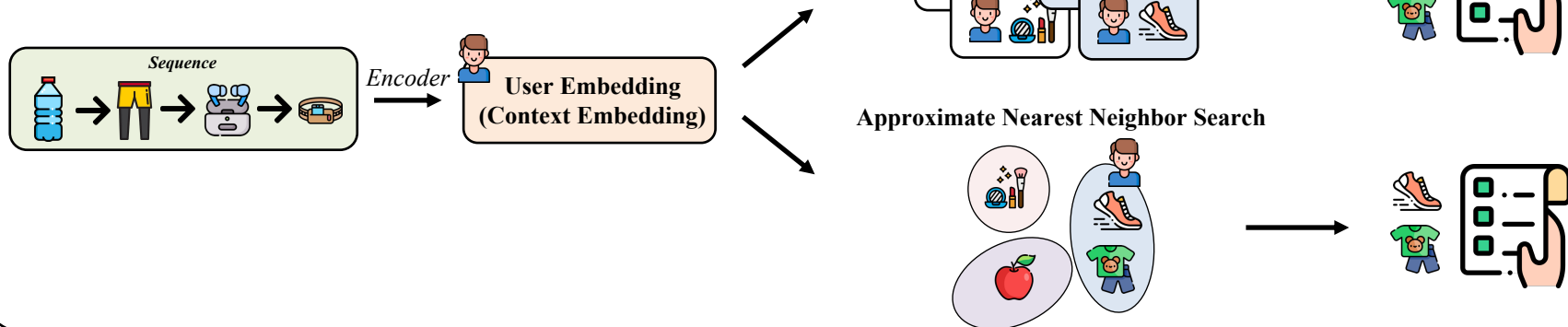
❖ Retrieve a small set of relevant candidates before applying a ranking model

- ❑ It is computationally impractical for a ranking model to evaluate hundreds of millions of items
- ❑ Conventional retrieval models often use a two-tower architecture
 - Separately encodes the user context and each item, then computes their similarity
- ❑ Since still expensive, Approximate Nearest Neighbor (ANN) search is commonly used

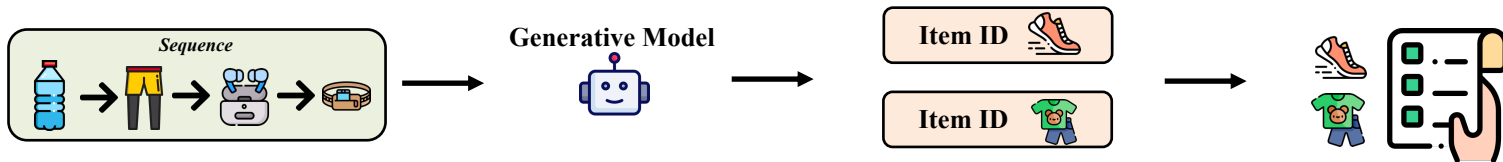
From Embedding Retrieval to Identifier Generation

Transformer Index for Generative Recommenders

Traditional Retrieval



Generative Retrieval



How TIGER Implements Generative Retrieval

Transformer Index for Generative Recommenders

❖ Identifier

- ❑ Replace atomic item IDs with content-based Semantic IDs

❖ Representation

- ❑ Discretize continuous content embeddings into multi-token Semantic IDs using RQ-VAE

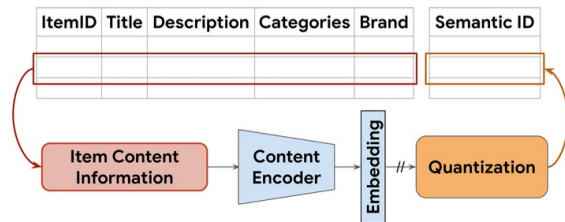
❖ Generation

- ❑ An encoder-decoder Transformer generates the next Semantic ID token by token

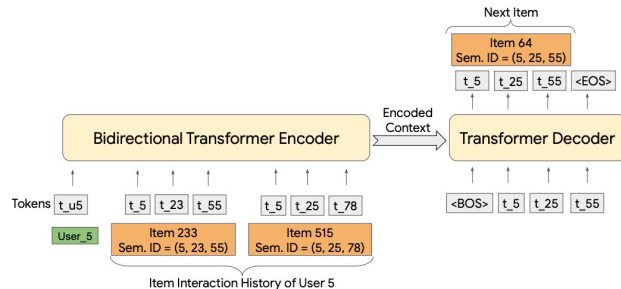
❖ Retrieval

- ❑ Map the generated Semantic ID back to the corresponding catalog item

Semantic ID Generation



Transformer-based Encoder-Decoder for Generative Retrieval



What Is a Semantic ID?

Transformer Index for Generative Recommenders

❖ Atomic Item ID vs. Semantic ID

Atomic Item ID



Pink shoes, Brand X
Atomic ID: 515



Orange shoes, Brand X
Atomic ID: 233



Orange shoes, Brand Y
Atomic ID: 64

Semantic Item ID



Pink shoes, Brand X
Semantic ID: (5, 25, 78)



Orange shoes, Brand X
Semantic ID: (5, 25, 55)



Orange shoes, Brand Y
Semantic ID: (5, 23, 55)

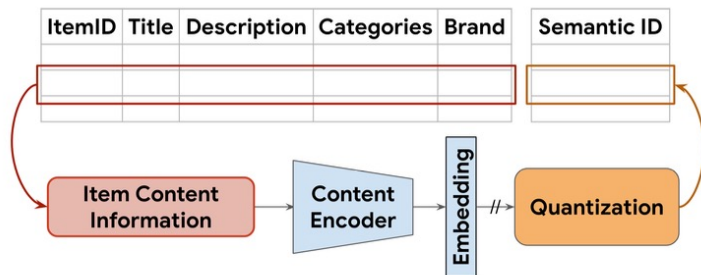
- ❑ An atomic ID is only a unique identifier
 - carries no meaning by itself
- ❑ ID 233 and ID 234 are not necessarily similar items

- ❑ Represents an item with multiple discrete tokens
- ❑ Similar items can share part of their Semantic IDs
 - not manually assigned a human-readable meaning

How Are Semantic IDs Created?

Transformer Index for Generative Recommenders

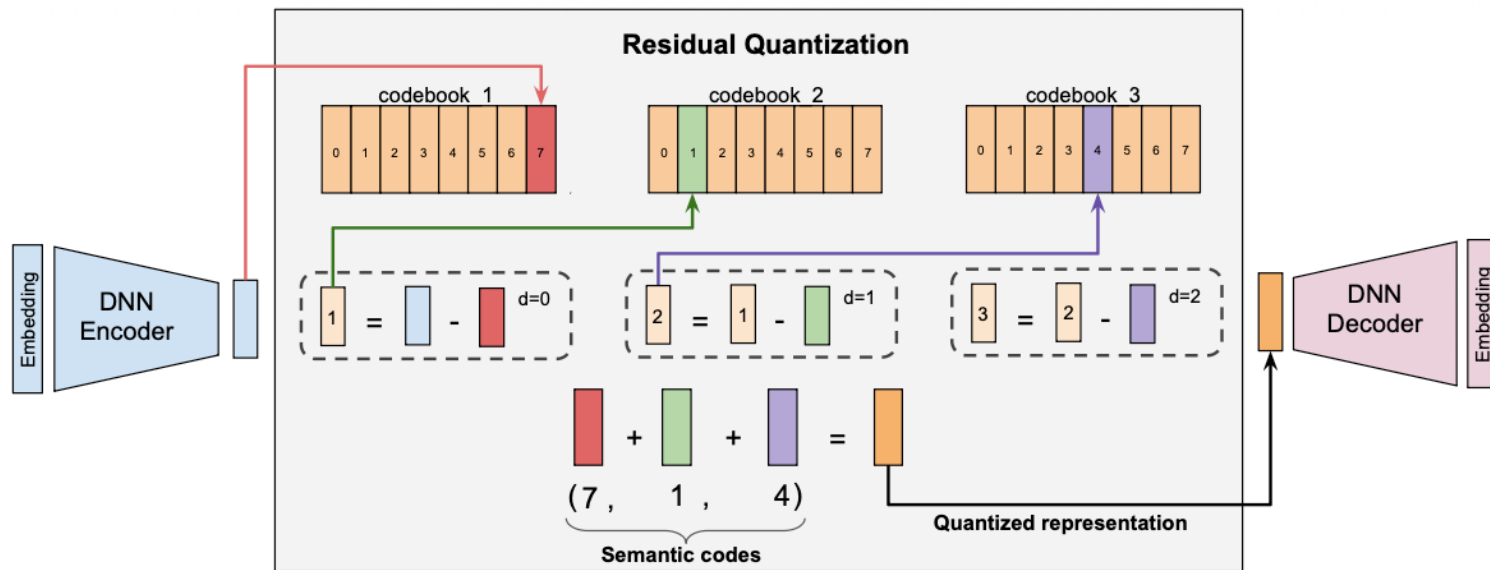
❖ Content-Based Semantic ID Generation



- ❑ Use item text features as input to a content encoder
 - Textual information such as title, description, and category
 - TIGER uses a pre-trained text encoder, Sentence-T5
- ❑ Convert the continuous embedding produced by content encoder into a sequence of discrete codes
 - The ordered sequence of codes becomes the item's Semantic ID

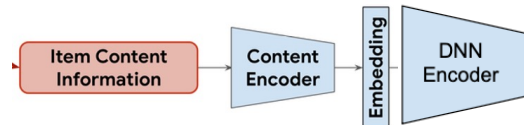
RQ-VAE for Semantic IDs

Transformer Index for Generative Recommenders



Methodology

Transformer Index for Generative Recommenders

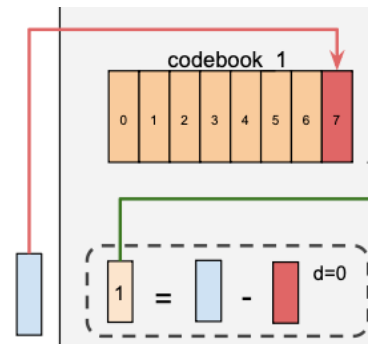


❖ Encoding Items from Their Content

- Content Encoder: Converts item content into a semantic embedding x
- RQ-VAE Encoder: Transforms x into a latent representation z suitable for quantization

❖ What Is a Codebook?

- Each codebook contains K learnable vectors
 - $C_d = \{e_0^{(d)}, e_1^{(d)}, \dots, e_{K-1}^{(d)}\}$
 - Each $e_k^{(d)}$ is a real-valued vector with the same dimension as z or its residual
- The numbers $0, 1, \dots, 7$ are indices of code vectors, not the vectors themselves
 - If z is closest to vector 7 in the first codebook, 7 becomes the first token of the Semantic ID

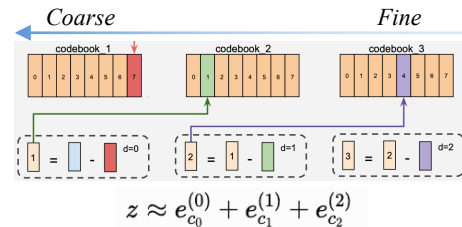


Methodology (Cont.)

Transformer Index for Generative Recommenders

❖ Why Are Multiple Codebooks Used?

- ❑ RQ-VAE doesn't represent the entire latent vector z with a single codebook
 - The first codebook captures the coarse structure of z
 - Each subsequent codebook quantizes the residual left by the previous stages
 - The representation is progressively refined from coarse to fine



❖ What Does the Decoder Do?

- ❑ The selected code vectors are summed to form the quantized latent representation
 - $\hat{z} = e_{c_0}^{(0)} + e_{c_1}^{(1)} + e_{c_2}^{(2)}$
- ❑ The decoder reconstructs the original content embedding x from $\hat{z}$
- ❑ The model is trained to minimize the reconstruction error
 - $\mathcal{L}_{\text{recon}} = \|x - \hat{x}\|^2$

Loss Term

Transformer Index for Generative Recommenders

❖ RQ-VAE Loss Term

- $\mathcal{L}(\mathbf{x}) := \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{rqvae}}$

- $\mathcal{L}_{\text{recon}} := \|\mathbf{x} - \hat{\mathbf{x}}\|^2$

- **Reconstruction Loss**
- Encourages the reconstructed embedding to preserve the semantic information of the original item embedding

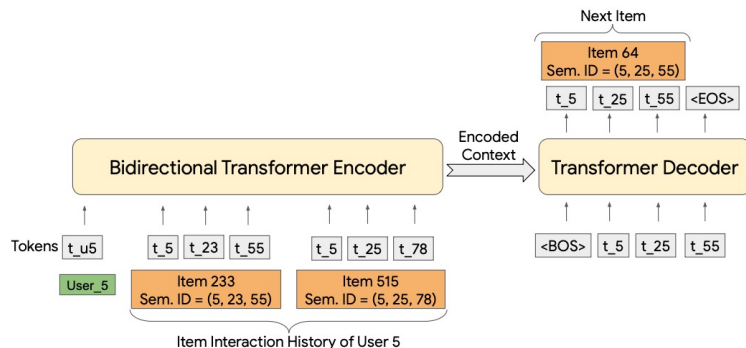
- $\mathcal{L}_{\text{rqvae}} := \sum_{i=0}^{m-1} \left(\|\text{sg}[\mathbf{r}_i] - \mathbf{e}_{c_i}\|^2 + \beta \|\mathbf{r}_i - \text{sg}[\mathbf{e}_{c_i}]\|^2 \right)$

- Aligns each residual with its selected code vector
- Stop-gradient separates codebook learning from encoder commitment

Generative Retrieval with Semantic IDs

Transformer Index for Generative Recommenders

- ❖ Predict the next item using a Transformer encoder–decoder architecture



- ❑ Convert each item in the user's interaction history into a Semantic ID token sequence
- ❑ Encode the interaction history and autoregressively generate the Semantic ID of the next item

Experiments

Transformer Index for Generative Recommenders



❖ Datasets

- ❑ Amazon Product Reviews
- ❑ Three real-world benchmark domains for sequential recommendation
 - Beauty
 - Sports and Outdoors
 - Toys and Games

❖ Evaluation Metrics

- ❑ Recall@K, NDCG@K ($K=\{5, 10\}$)

Table 6: Dataset statistics for the three real-world benchmarks.

Dataset	# Users	# Items	Sequence Length	
			Mean	Median
Beauty	22,363	12,101	8.87	6
Sports and Outdoors	35,598	18,357	8.32	6
Toys and Games	19,412	11,924	8.63	6

Performance on Sequential Recommendation

Transformer Index for Generative Recommenders

Methods	Sports and Outdoors				Beauty				Toys and Games			
	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10
P5 [8]	0.0061	0.0041	0.0095	0.0052	0.0163	0.0107	0.0254	0.0136	0.0070	0.0050	0.0121	0.0066
Caser [33]	0.0116	0.0072	0.0194	0.0097	0.0205	0.0131	0.0347	0.0176	0.0166	0.0107	0.0270	0.0141
HGN [25]	0.0189	0.0120	0.0313	0.0159	0.0325	0.0206	0.0512	0.0266	0.0321	0.0221	0.0497	0.0277
GRU4Rec [11]	0.0129	0.0086	0.0204	0.0110	0.0164	0.0099	0.0283	0.0137	0.0097	0.0059	0.0176	0.0084
BERT4Rec [32]	0.0115	0.0075	0.0191	0.0099	0.0203	0.0124	0.0347	0.0170	0.0116	0.0071	0.0203	0.0099
FDSA [42]	0.0182	0.0122	0.0288	0.0156	0.0267	0.0163	0.0407	0.0208	0.0228	0.0140	0.0381	0.0189
SASRec [17]	0.0233	0.0154	0.0350	0.0192	0.0387	0.0249	0.0605	0.0318	0.0463	0.0306	0.0675	0.0374
S ³ -Rec [44]	0.0251	0.0161	0.0385	0.0204	0.0387	0.0244	0.0647	0.0327	0.0443	0.0294	0.0700	0.0376
TIGER [Ours]	0.0264	0.0181	0.0400	0.0225	0.0454	0.0321	0.0648	0.0384	0.0521	0.0371	0.0712	0.0432
	+5.22%	+12.55%	+3.90%	+10.29%	+17.31%	+29.04%	+0.15%	+17.43%	+12.53%	+21.24%	+1.71%	+14.97%

□ Full-catalog retrieval rather than sampled ranking evaluation

- ▶ A common sampled-ranking protocol evaluates one target item against 100 sampled negative items
- ▶ In the retrieval setting, baselines construct a user or context embedding and item embeddings

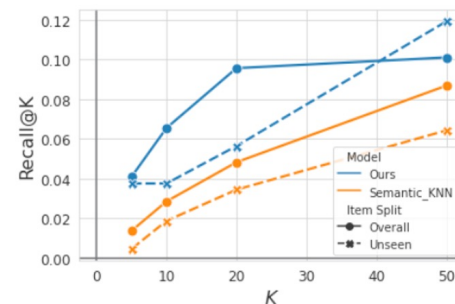
New Capabilities: Cold Start

Transformer Index for Generative Recommenders

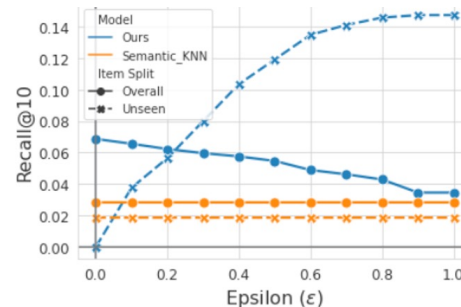
- Newly added items have no user interaction history in the training data
 - Atomic-ID-based models assign them arbitrary identifiers
 - This makes it difficult to retrieve them as potential recommendations

- Remove 5% of the test items from the training data
 - These items are treated as unseen items
 - If the model generates the Semantic ID assigned to an unseen item, that item can still be included in the retrieval results
 - The maximum proportion of unseen items in the Top- K results is controlled by ϵ

- TIGER achieves higher recall on unseen items than Semantic KNN



(a) Recall@K vs. K, ($\epsilon = 0.1$).



(b) Recall@10 vs. ϵ .

New Capabilities: Controllable Diversity

Transformer Index for Generative Recommenders

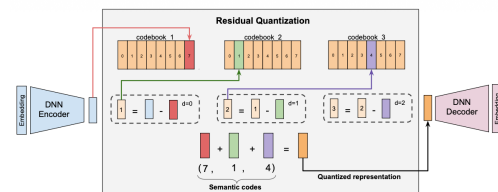
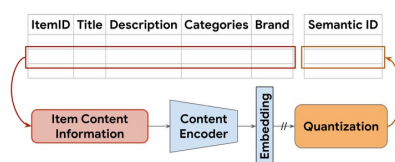
Temperature	Entropy@10	Entropy@20	Entropy@50
T = 1.0	0.76	1.14	1.70
T = 1.5	1.14	1.52	2.06
T = 2.0	1.38	1.76	2.28

Target Category	Most-common Categories for top-10 predicted items	
	T = 1.0	T = 2.0
Hair Styling Products	Hair Styling Products	Hair Styling Products, Hair Styling Tools, Skin Face
Tools Nail	Tools Nail	Tools Nail, Makeup Nails
Makeup Nails	Makeup Nails	Makeup Nails, Skin Hands & Nails, Tools Nail
Skin Eyes	Skin Eyes	Hair Relaxers, Skin Face, Hair Styling Products, Skin Eyes
Makeup Face	Tools Makeup Brushes, Makeup Face	Tools Makeup Brushes, Makeup Face, Skin Face, Makeup Sets, Hair Styling Tools
Hair Loss Products	Hair Loss Products, Skin Face, Skin Body	Skin Face, Hair Loss Products, Hair Shampoos, Hair & Scalp Treatments, Hair Conditioners

- ❑ Control recommendation diversity using temperature-based sampling during decoding
 - A higher temperature flattens the token probability distribution and increases diversity
 - Sampling earlier Semantic ID tokens can move the recommendation toward a broader semantic region
 - Sampling later tokens can produce different items within a similar semantic region

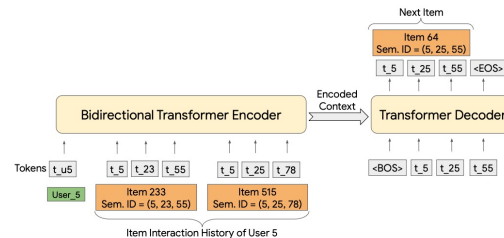
Conclusion (TIGER)

Transformer Index for Generative Recommenders



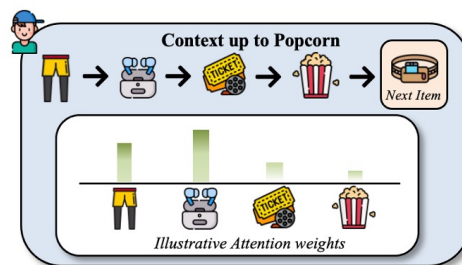
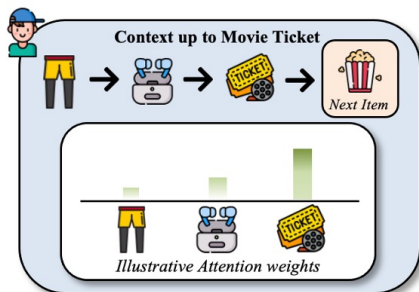
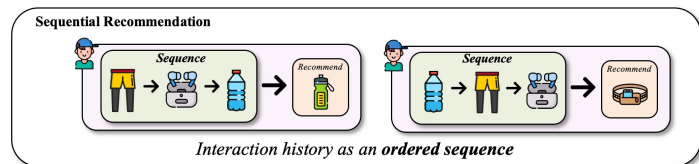
CAU

- Reframes retrieval-stage next-item recommendation from embedding-based item search as direct Semantic ID generation
- Represents each item with hierarchical Semantic IDs, generated by quantizing content embeddings through RQ-VAE
- Autoregressively generates the next item's Semantic ID using an encoder-decoder Transformer
- Outperforms conventional sequential recommenders on three Amazon benchmarks in Recall and NDCG
- Enables additional capabilities such as cold-start recommendation and controllable recommendation diversity

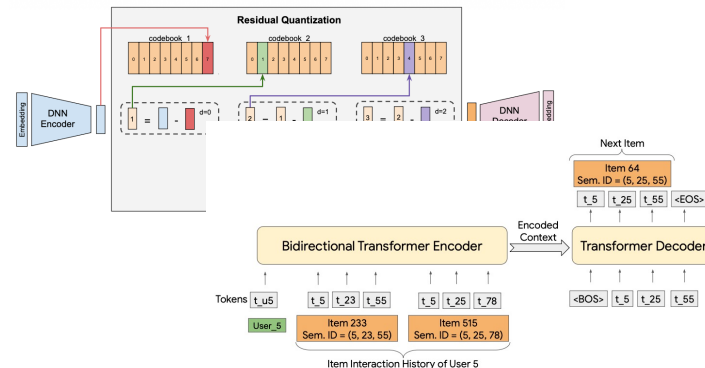


Conclusion

Sequential Recommendation: From Markov Transitions to Generative Retrieval

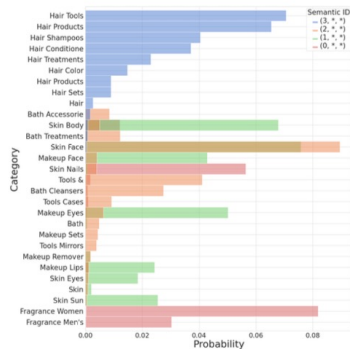
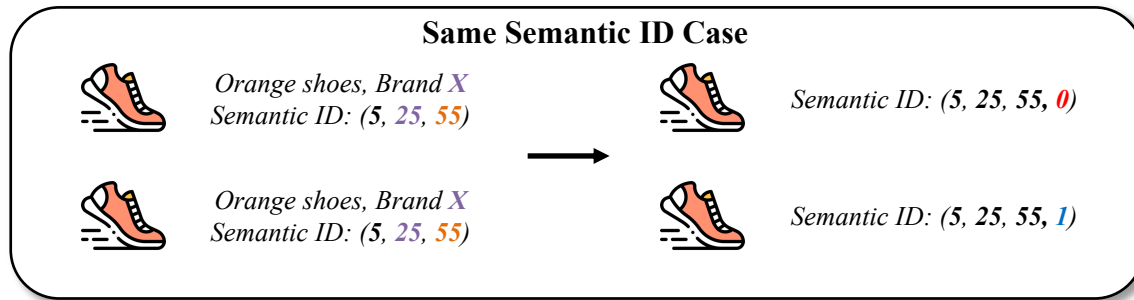


Model	How Past Interactions are Modeled	Strengths	Key Limitations
FPMC	Recent item transitions	Simple and efficient	Limited long-range dependency modeling
GRU4Rec	Sequential hidden states	Captures the entire session	Sequential computation; early information may fade
Caser	CNN patterns within a fixed window	Captures local and union-level patterns	Limited receptive field

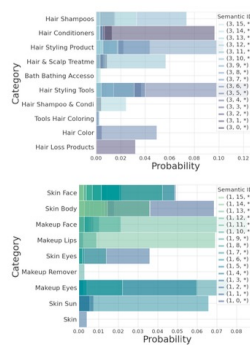




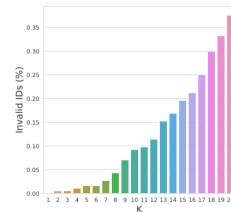
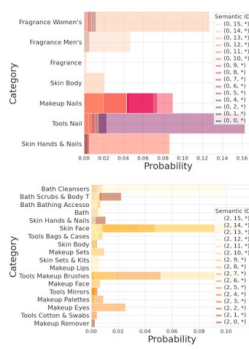
Appendix



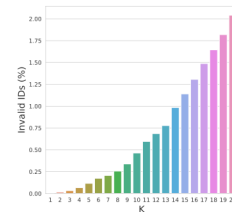
(a) The ground-truth category distribution for all the items in the dataset colored by the value of the first codeword c_1 .



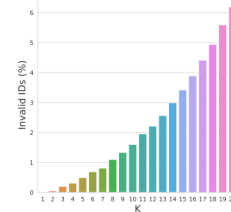
(b) The category distributions for items having the Semantic ID as $(c_1, *, *)$, where $c_1 \in \{1, 2, 3, 4\}$. The categories are color-coded based on the second semantic token c_2 .



(a) Sports and Outdoors



(b) Beauty



(c) Toys and Games

Figure 6: Percentage of invalid IDs when generating Semantic IDs using Beam search for various values of K . As shown, $\sim 0.3\% - 6\%$ of the IDs are invalid when retrieving the top-20 items.